

Functions, Organization, Arrays, Vectors, Lists and some comparison

Unit-01

Functions

Unit-01.1

cpp

 Copy code

```
return_type function_name(parameter_list) {  
    // Function body  
    // Statements and logic go here  
    return return_value; // Optional, depends on the return type  
}
```

Here's a breakdown of each part:

- **return_type:** The data type of the value that the function returns. It can be `void` if the function doesn't return any value.
- **function_name:** The name of the function, following C++ naming conventions.
- **parameter_list:** The list of parameters (input values) that the function accepts. It can be empty if the function takes no parameters.
- **Function body:** The block of code that defines what the function does. This is where you write the logic of your function.
- **return_value:** The value that the function returns. This is optional for functions with a return type of `void` but required for other types.

cpp

[A]

```
int add(int a, int b) {  
    return a + b;  
}
```

cpp

[B]

```
void printMessage(std::string message) {  
    std::cout << message << std::endl;  
}
```

cpp

[C]

```
void greet() {  
    std::cout << "Hello, world!" << std::endl;  
}
```

cpp

[D]

```
double square(double x) {  
    return x * x;  
}
```

SUMMARY

1. Function with arguments no return type
2. Function with arguments with return type
3. Function without arguments with return type
4. Function without arguments without return type

Standard Functions:

- These are functions provided by the C++ standard library.
- Examples include functions like `printf`, `scanf`, `cin`, `cout`, etc.

cpp

 Copy code

```
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

User-Defined Functions:

- Functions defined by the user to perform specific tasks.
- Examples include functions for calculations, processing, and modularizing code.

cpp

 Copy code

```
#include <iostream>

// User-defined function to add two numbers
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(3, 4);
    std::cout << "Result: " << result << std::endl;
    return 0;
}
```

Recursive Functions:

- Functions that call themselves either directly or indirectly.
- Useful for solving problems that can be broken down into smaller instances of the same problem.

VARIOUS TYPES OF FUNCTIONS

3. *Recursive Functions*

cpp

 Copy code

```
#include <iostream>

// Recursive function to calculate factorial
int factorial(int n) {
    if (n <= 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int result = factorial(5);
    std::cout << "Factorial: " << result << std::endl;
    return 0;
}
```

Inline Functions:

- Functions that are expanded in line when called, rather than executing a function call.
- Helps in reducing the overhead of function calls for small tasks.

cpp

 Copy code

```
#include <iostream>

// Inline function to calculate square
inline int square(int x) {
    return x * x;
}

int main() {
    int result = square(4);
    std::cout << "Square: " << result << std::endl;
    return 0;
}
```



VARIOUS TYPES OF FUNCTIONS

4. *Inline Functions...*

C++

 Copy

```
// Inline_Member_Functions.cpp
class Account
{
public:
    Account(double initial_balance) { balance = initial_balance; }
    double GetBalance();
    double Deposit( double Amount );
    double Withdraw( double Amount );
private:
    double balance;
};

inline double Account::GetBalance()
{
    return balance;
}

inline double Account::Deposit( double Amount )
{
    return ( balance += Amount );
}

inline double Account::Withdraw( double Amount )
{
    return ( balance -= Amount );
}

int main()
{
}
```

In the following class declaration, the Account constructor is an inline function because it is defined in the body of the class declaration. The member functions ***GetBalance***, Deposit, and Withdraw are specified inline in their definitions. The inline keyword is optional in the function declarations in the class declaration.

VARIOUS TYPES OF FUNCTIONS

4. Inline Functions...

```
1  #include <iostream>
2
3  #define mult(a, b) a * b
4
5  inline int multiply(int a, int b)
6  {
7      return a * b;
8  }
9
10 int main()
11 {
12     std::cout << mult(5 + 5, 5 + 5) << std::endl;
13     std::cout << multiply(5 + 5, 5 + 5) << std::endl;
14
15     std::cout << mult(2, 2.2) << std::endl;
16     std::cout << multiply(2, 2.2);
17 }
```

Can you guess the output from

- L12

- L13

-L15

-L16

VARIOUS TYPES OF FUNCTIONS

4. Inline Functions vs Macros

```
1  #include <iostream>
2
3  #define mult(a, b) a * b
4
5  inline int multiply(int a, int b)
6  {
7      return a * b;
8  }
9
10 int main()
11 {
12     std::cout << mult(5 + 5, 5 + 5) << std::endl; // outputs 35
13     std::cout << multiply(5 + 5, 5 + 5) << std::endl; // outputs 100
14
15     // no warning
16     std::cout << mult(2, 2.2) << std::endl;
17     /* Warning C4244
18     'argument': conversion from 'double' to 'int', possible loss of data*/
19     std::cout << multiply(2, 2.2);
20 }
```

Here are some of the differences between the macro and the inline function:

- Macros are always expanded inline. However, an inline function is only inlined when the compiler determines it is the optimal thing to do.
- The macro may result in unexpected behavior. For example, the macro `mult(5+5,5+5)` expands to `5 + 5 * 5 + 5` resulting in 35, whereas the function evaluates `10 * 10`. You could address that by defining the macro as `#define mult(a, b) ((a)*(b))`.
- An inline function is subject to semantic processing by the compiler, whereas the preprocessor expands macros without that same benefit. Macros aren't type-safe, whereas functions are.

Function Overloading:

- Defining multiple functions with the same name but different parameter lists.
- Allows using the same function name for operations with different types or numbers of parameters.

VARIOUS TYPES OF FUNCTIONS

4. *Function Overloading*

cpp

 Copy code

```
#include <iostream>

// Function overloading
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int main() {
    std::cout << "Sum (int): " << add(3, 4) << std::endl;
    std::cout << "Sum (double): " << add(3.5, 4.5) << std::endl;
    return 0;
}
```

Lambda Functions (C++11 and later):

- Anonymous functions defined using the lambda expression.
- Convenient for short, one-time use functions.

cpp

 Copy code

```
#include <iostream>

int main() {
    // Lambda function to add two numbers
    auto add = [](int a, int b) -> int {
        return a + b;
    };

    int result = add(3, 4);
    std::cout << "Result: " << result << std::endl;
    return 0;
}
```

FUNCTION CALL

Call by Value

cpp

Copy code

```
#include <iostream>

// Function that adds a value to a number (call by value)
void addByValue(int num, int value) {
    num += value;
}

int main() {
    int number = 5;
    int increment = 3;

    std::cout << "Before addByValue: number = " << number << std::endl;

    // Calling the function by value
    addByValue(number, increment);

    std::cout << "After addByValue: number = " << number << std::endl;

    return 0;
}
```

The function call has **NO effect** on ***number*** inside main function ***i.e., number is still 5***

FUNCTION CALL

Call by Value

```
#include <iostream>

// Function that adds a value to a number (call by reference)
void addByReference(int &num, int value) {
    num += value;
}

int main() {
    int number = 5;
    int increment = 3;

    std::cout << "Before addByReference: number = " << number << std::endl;

    // Calling the function by reference
    addByReference(number, increment);

    std::cout << "After addByReference: number = " << number << std::endl;

    return 0;
}
```

The function call **has effect** on ***number*** inside main function ***i.e., number will be 8***

In C++, you can provide default values for function parameters, allowing the caller to omit those arguments if they want to use the default values. Here's an example of a function with default arguments:

cpp

 Copy code

```
#include <iostream>

// Function with default arguments
void printRectangle(int length = 5, int width = 3) {
    std::cout << "Rectangle: Length = " << length << ", Width = " << width << "\n";
    std::cout << "Area: " << length * width << std::endl;
}

int main() {
    // Calling the function with default arguments
    printRectangle();

    // Calling the function with custom arguments
    printRectangle(8, 6);

    return 0;
}
```

Explanation:

- The `printRectangle` function has two parameters, `length` and `width`, with default values of `5` and `3` respectively.
- When the function is called without arguments (`printRectangle();`), it uses the default values.
- When the function is called with custom arguments (`printRectangle(8, 6);`), it overrides the default values.

Local Scope:

- Variables declared inside a block of code, such as within a function or a compound statement, have local scope.
- They are only accessible within that specific block or function.
- Local variables are created when the block is entered and destroyed when the block is exited.

cpp

 Copy code

```
#include <iostream>

void exampleFunction() {
    int localVar = 5; // Local variable
    std::cout << "Local variable: " << localVar << std::endl;
}

int main() {
    exampleFunction();
    // Uncommenting the line below would result in a compilation error
    // std::cout << "Trying to access local variable: " << localVar <<
    return 0;
}
```

Global Scope:

- Variables declared outside of any function or block have global scope.
- They are accessible from any part of the program, including all functions.
- Global variables persist throughout the program's execution.

cpp

 Copy code

```
#include <iostream>

int globalVar = 10; // Global variable

void exampleFunction() {
    std::cout << "Global variable from function: " << globalVar << std::endl;
}

int main() {
    std::cout << "Global variable from main: " << globalVar << std::endl;
    exampleFunction();
    return 0;
}
```

Function Parameters:

- Parameters passed to a function have a scope within that function.
- They act like local variables within the function and are only accessible within the function body.
- Parameter names in a function declaration act as local variables.

VARIABLE SCOPE
Parameters

cpp

 Copy code

```
#include <iostream>

void printSum(int a, int b) {
    int sum = a + b; // sum is a local variable
    std::cout << "Sum: " << sum << std::endl;
}

int main() {
    printSum(3, 4);
    // Uncommenting the line below would result in a compilation error
    // std::cout << "Trying to access parameter 'a': " << a << std::endl;
    return 0;
}
```

Organization

Unit-01.2

As your project gets bigger and bigger, it is impossible to put everything in one big file.

WHY?

EXPLORER ...

> OPEN EDITORS

EXTERNEEXAMPLE

main

main.cpp

C++ main.cpp X

C++ main.cpp > main()

```
1  #include <iostream>
2
3  bool isOdd(int num){
4      /* Returns True if num is odd else False */
5      return (num % 2) != 0;
6  }
7
8  int main()
9  {
10     int num;
11     std::cout << "Enter: ";
12     std::cin >> num;
13     if(isOdd(num))
14         std::cout << "ODD" << std::endl;
15     else
16         std::cout << "NOT ODD" << std::endl;
17     return 0;
18 }
19
20
21
```

PROBLEMS

TERMINAL

DEBUG CONSOLE

PORTS

COMMENTS

```
> g++ -o main main.cpp
> ./main
Enter: 33
ODD
```

Organization

Simple easy: good for small projects

CASE1: Everything in one file, function first and main

EXPLORER ...

> OPEN EDITORS

✓ EXTERNEXAMPLE

main

main.cpp

C++ main.cpp ×

C++ main.cpp > isOdd(int)

```
1  #include <iostream>
2
3  bool isOdd(int);
4
5  int main()
6  {
7      int num;
8      std::cout << "Enter: ";
9      std::cin >> num;
10     if(isOdd(num))
11     |     std::cout << "ODD" << std::endl;
12     else
13     |     std::cout << "NOT ODD" << std::endl;
14     return 0;
15 }
16
17 bool isOdd(int num){
18     /* Returns True if num is odd else False */
19     return (num % 2) != 0;
20 }
21
22
```

PROBLEMS

TERMINAL

DEBUG CONSOLE

PORTS

COMMENTS

```
> g++ -o main main.cpp
> ./main
Enter: 33
ODD
```

Organization

Simple easy: good for small projects

CASE1: Everything in one file, with function declaration

```

C++ main.cpp × ...
C++ main.cpp > ...
1  #include <iostream>
2
3  extern bool isOdd(int);
4
5  int main()
6  {
7      int num;
8      std::cout << "Enter: ";
9      std::cin >> num;
10     if(isOdd(num))
11     |     std::cout << "ODD" << std::endl;
12     else
13     |     std::cout << "NOT ODD" << std::endl;
14     return 0;
15 }
16
17

```

```

C++ utils.cpp ×
C++ utils.cpp > isOdd(int)
1  bool isOdd(int num){
2      /* Returns True if num is odd
3      |     return (num % 2) != 0;
4      | }

```

PROBLEMS TERMINAL DEBUG CONSOLE PORTS COMMENTS

```

> g++ -o main main.cpp utils.cpp
> ./main
Enter: 33
ODD

```

Organization

Your project is now a big deal!!

CASE2: USING External Function from different file

Organization

Your project is now a big deal!!

CASE2: USING Header Files

```
C++ main.cpp × ...
C++ main.cpp > ...
1  #include <iostream>
2  #include "utils.h"
3
4  int main()
5  {
6      int num;
7      std::cout << "Enter: ";
8      std::cin >> num;
9      if(isOdd(num))
10         std::cout << "ODD" << std::endl;
11     else
12         std::cout << "NOT ODD" << std::endl;
13     return 0;
14 }
15
16

h utils.h ×
h utils.h > ...
1  /* utils header file */
2  #ifndef UTILS_H
3  #define UTILS_H
4
5  bool isOdd(int);
6
7  #endif // UTILS_H

C++ utils.cpp ×
C++ utils.cpp > ...
1  /* utils cpp file */
2  #include "utils.h"
3
4  bool isOdd(int num){
5      /* Returns True if num is odd else False */
6      return (num % 2) != 0;
7  }

PROBLEMS  TERMINAL  DEBUG CONSOLE  PORTS  COMMENTS
> g++ main.cpp utils.cpp -o main
> ./main
Enter: 33
ODD
```

Organization
BUT WHY?

BUT WHY?

Organization

Case Study

C++ main.cpp

```
C++ main.cpp > main()
1  #include <iostream>
2  #include "utils.h"
3  #include "factory.h"
4
5  int main()
6  {
7      int num;
8      std::cout << "Enter: ";
9      std::cin >> num;
10     // Here we are using utils
11     if(Helpers::isOdd(num))
12     |     std::cout << "ODD" << std::endl;
13     else
14     |     std::cout << "NOT ODD" << std::endl;
15     // Here we are making factory call
16     Toolkit::smileIfEven(num);
17     return 0;
18 }
19
```

PROBLEMS

TERMINAL

DEBUG CONSOLE

```
> g++ -c utils.cpp
> g++ -c factory.cpp
> g++ -c main.cpp
> g++ utils.o factory.o main.o -o main
> ./main
Enter: 33
ODD
> ./main
Enter: 32
NOT ODD
HiHiHiHiHiHi
~/Desktop/Cpp2308/ExternExample >
```

h utils.h

```
h utils.h > {} Helpers
1  /* utils header file */
2  #ifndef UTILS_H
3  #define UTILS_H
4
5  namespace Helpers {
6      bool isOdd(int);
7  }
8
9  #endif // UTILS_H
```

C++ utils.cpp

```
C++ utils.cpp > {} Helpers
1  /* utils cpp file */
2  #include "utils.h"
3
4  namespace Helpers {
5      bool isOdd(int num){
6          /* Returns True if num is
7           *
8           */
9      }
```

h factory.h

```
h factory.h > ...
1  /* factory header file */
2  #ifndef FACTORY_H
3  #define FACTORY_H
4
5  namespace Toolkit {
6      void smileIfEven(int);
7  }
8
9  #endif // FACTORY_H
```

C++ factory.cpp

```
C++ factory.cpp > {} Toolkit > smileIfEven(int)
1  /* utils cpp file */
2  #include <iostream>
3  #include "factory.h"
4  #include "utils.h"
5
6  namespace Toolkit {
7      void smileIfEven(int num){
8          if(!Helpers::isOdd(num))
9          |
10         |     std::cout << "HiHiHiHiHiHi"
11         |     << std::endl;
12     }
```

Organization

Having multiple files, work independently, compile only that changes

```
C++ main.cpp > main()
1  #include <iostream>
2  #include "utils.h"
3  #include "factory.h"
4
5  int main()
6  {
7      int num;
8      std::cout << "Enter: ";
9      std::cin >> num;
10     // Here we are using utils
11     if(Helpers::isOdd(num))
12     |     std::cout << "ODD" << std::endl;
13     else
14     |     std::cout << "NOT ODD" << std::endl;
15     // Here we are making factory call
16     Toolkit::smileIfEven(num);
17     return 0;
18 }
19
```

```
h utils.h > {} Helpers
1  /* utils header file */
2  #ifndef UTILS_H
3  #define UTILS_H
4
5  namespace Helpers {
6      bool isOdd(int);
7  }
8
9  #endif // UTILS_H
```

```
h factory.h > ...
1  /* factory header file */
2  #ifndef FACTORY_H
3  #define FACTORY_H
4
5  namespace Toolkit {
6      void smileIfEven(int);
7  }
8
9  #endif // FACTORY_H
```

```
C++ utils.cpp > {} Helpers
1  /* utils cpp file */
2  #include "utils.h"
3
4  namespace Helpers {
5      bool isOdd(int num){
6          /* Returns True if num is odd */
7          return (num % 2) != 0;
8      }
9  }
```

```
C++ factory.cpp > {} Toolkit > smileIfEven(int)
1  /* utils cpp file */
2  #include <iostream>
3  #include "factory.h"
4  #include "utils.h"
5
6  namespace Toolkit {
7      void smileIfEven(int num){
8          if(!Helpers::isOdd(num))
9              std::cout << "HAHAHAHAHAHA"
10             << std::endl;
11      }
12 }
```

```
PROBLEMS  TERMINAL  DEBUG CONSOLE  PORTS  COMMENT
> g++ -c utils.cpp
> g++ -c factory.cpp
> g++ -c main.cpp
> g++ utils.o factory.o main.o -o main
> ./main
Enter: 33
ODD
> ./main
Enter: 32
NOT ODD
HiHiHiHiHiHi
> g++ -c factory.cpp
> g++ utils.o factory.o main.o -o main
> ./main
Enter: 33
ODD
> ./main
Enter: 32
NOT ODD
HAHAHAHAHAHA
```

```
namespace Toolkit {
    void smileIfEven(int num){
        if(!Helpers::isOdd(num))
            std::cout << "HiHiHiHiHiHi"
            << std::endl;
    }
}
```

Arrays

Unit-01.3

Character Array:

ARRAY
Char Array

cpp

 Copy code

```
#include <iostream>

int main() {
    // Creating and initializing a character array
    char charArray[] = {'H', 'e', 'l', 'l', 'o', '\0'}; // Null-terminator
    std::cout << "Character Array: " << charArray << std::endl;

    return 0;
}
```

Explanation:

- In this example, a character array named `charArray` is created and initialized with the characters 'H', 'e', 'l', 'l', 'o', and the null character `\0`.
- The null character is added to make it a null-terminated string, allowing us to use it with functions that expect strings.
- The array is then printed, resulting in the output "Hello".

Number Array:

ARRAY

Number Array

cpp

 Copy code

```
#include <iostream>

int main() {
    // Creating and initializing a number array
    int numArray[] = {1, 2, 3, 4, 5};
    std::cout << "Number Array: ";

    // Printing elements of the array
    for (int i = 0; i < 5; ++i) {
        std::cout << numArray[i] << " ";
    }

    std::cout << std::endl;

    return 0;
}
```

Explanation:

- This example creates an integer array named `numArray` and initializes it with the values 1, 2, 3, 4, and 5.
- The elements of the array are then printed using a loop, resulting in the output "Number Array: 1 2 3 4 5".

Struct Array:

ARRAY Struct Array

cpp

Copy code

```
#include <iostream>
#include <string>

// Define a struct
struct Person {
    std::string name;
    int age;
};

int main() {
    // Creating and initializing a struct array
    Person people[] = {"Alice", 25}, {"Bob", 30}, {"Charlie", 22};
    std::cout << "Struct Array:" << std::endl;

    // Printing elements of the array
    for (const auto& person : people) {
        std::cout << "Name: " << person.name << ", Age: " << person.age << std::endl;
    }

    return 0;
}
```

PLEASE IGNORE THIS
FOR LOOP FOR NOW

Explanation:

- This example defines a struct named `Person` with members `name` (a string) and `age` (an integer).
- An array of `Person` structs named `people` is created and initialized with three instances of the struct, each representing a person with a name and age.
- The elements of the array are then printed using a range-based for loop, resulting in the output:

yaml

Copy code

```
Struct Array:
Name: Alice, Age: 25
Name: Bob, Age: 30
Name: Charlie, Age: 22
```

Passing arrays as function arguments in C++ involves using pointers or references.

Here's an example of passing an array as a function argument using pointers:

ARRAY

As function argument

cpp

 Copy code

```
#include <iostream>

// Function that takes an array and its size as arguments
void printArray(int arr[], int size) {
    for (int i = 0; i < size; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};

    std::cout << "Array in main: ";
    printArray(numbers, 5); // Passing array to the function

    return 0;
}
```

Explanation:

- The `printArray` function takes two parameters: an array (`arr`) and its size (`size`).
- Inside the function, a loop is used to print each element of the array.
- In the `main` function, an array named `numbers` is declared and initialized with values.
- The `printArray` function is then called, passing the array `numbers` and its size (`5`) as arguments.
- The output will be "Array in main: 1 2 3 4 5".

Note: When passing an array to a function, it decays into a pointer to its first element. The size of the array is often passed separately to indicate the number of elements.

Passing arrays as function arguments in C++ involves using pointers or references.

Here's an example of passing an array as a function argument using pointers:

ARRAY

As function argument

cpp

 Copy code

```
#include <iostream>

// Function that takes an array and its size as arguments
void printArray(int arr[], int size) {
    for (int i = 0; i < size; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};

    std::cout << "Array in main: ";
    printArray(numbers, 5); // Passing array to the function

    return 0;
}
```

Explanation:

- The `printArray` function takes two parameters: an array (`arr`) and its size (`size`).
- Inside the function, a loop is used to print each element of the array.
- In the `main` function, an array named `numbers` is declared and initialized with values.
- The `printArray` function is then called, passing the array `numbers` and its size (`5`) as arguments.
- The output will be "Array in main: 1 2 3 4 5".

Note: When passing an array to a function, it decays into a pointer to its first element. The size of the array is often passed separately to indicate the number of elements.

*WHAT HAPPENS IF YOU MODIFY
ARRAY INSIDE FUNCTION?*

ARRAY

You can also pass ref.

If you prefer to use references, you can modify the function accordingly:

cpp

 Copy code

```
#include <iostream>

// Function that takes an array and its size as arguments using references
void printArray(const int (&arr)[], int size) {
    for (int i = 0; i < size; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};

    std::cout << "Array in main: ";
    printArray(numbers, 5); // Passing array to the function

    return 0;
}
```

In this version, the function `printArray` takes a reference to an array. The `const` ensures that the function doesn't modify the elements of the array. The choice between using pointers or references depends on the specific requirements of your code.

Declaration and Initialization:

- You can declare and initialize arrays using the appropriate syntax.

cpp

 Copy code

```
int numbers[5] = {1, 2, 3, 4, 5};
```

Accessing Elements:

- You can access individual elements of an array using the index.

cpp

 Copy code

```
int value = numbers[2]; // Accessing the third element
```

Iteration:

- You can iterate over the elements of an array using loops.

cpp

 Copy code

```
for (int i = 0; i < 5; ++i) {  
    // Process each element  
}
```

Passing to Functions:

- You can pass arrays to functions as arguments.

cpp

 Copy code

```
void processArray(int arr[], int size) {  
    // Function logic  
}  
  
// Call the function  
processArray(numbers, 5);
```

cpp

 Copy code

ARRAY

WHAT IS ALLOWED?

```
#include <iostream>

// Function that creates and returns an array (pointer to its first element)
int* createArray(int size) {
    int* newArray = new int[size]; // Dynamic array creation
    for (int i = 0; i < size; ++i) {
        newArray[i] = i + 1; // Initialize values (1, 2, 3, ...)
    }
    return newArray; // Return the pointer to the first element
}

int main() {
    int size = 5;

    // Call the function to create an array
    int* returnedArray = createArray(size);

    // Access and print elements of the returned array
    std::cout << "Returned Array: ";
    for (int i = 0; i < size; ++i) {
        std::cout << returnedArray[i] << " ";
    }

    // Don't forget to delete the dynamically allocated array to avoid memory leak
    delete[] returnedArray;

    return 0;
}
```

Explanation:

- The `createArray` function dynamically allocates an array of integers and initializes its values.
- It returns the pointer to the first element of the array.
- In the `main` function, the returned array is accessed, and its elements are printed.
- It's important to use `delete[]` to free the memory allocated for the array when it's no longer needed.



Dynamic Size:

- Arrays in C++ have a fixed size determined at compile-time. You cannot dynamically change the size of an array.

Bounds Checking:

- C++ does not perform automatic bounds checking. Accessing an array out of its bounds leads to undefined behavior.

Copying and Assignment:

- You cannot directly assign or copy one array to another using the assignment operator.

ARRAY

WHAT IS NOT ALLOWED?

cpp

 Copy code

```
int newArray[5];  
// newArray = numbers; // Error:
```

Returning Arrays from Functions:

- Directly returning arrays from functions is not allowed. You can return a pointer or use containers like `std::vector` for dynamic behavior.

cpp

 Copy code

```
int* createArray() {  
    int arr[5] = {1, 2, 3, 4, 5};  
    // return arr; // Error: Returning address of local variable  
    return nullptr;  
}
```

Array Iteration

Unit-01.3.1

```
#include <iostream>

int main() {
    // Declare and initialize an array
    int myArray[] = {1, 2, 3, 4, 5};

    // Get the size of the array
    int arraySize = sizeof(myArray) / sizeof(myArray[0]);

    // Use a for loop to iterate through the array
    for (int i = 0; i < arraySize; ++i) {
        std::cout << "Element at index " << i << ": " << myArray[i] <<
    }

    return 0;
}
```

cpp

RANGE BASED

 Copy code

```
#include <iostream>

int main() {
    int myArray[] = {1, 2, 3, 4, 5};

    // Use a range-based for loop to iterate through the array
    for (int value : myArray) {
        std::cout << value << "\n";
    }

    return 0;
}
```

String Array:

ARRAY

String Array

cpp

 Copy code

```
#include <iostream>
#include <string>

int main() {
    // Creating and initializing a string array
    std::string strArray[] = {"Apple", "Banana", "Orange"};
    std::cout << "String Array: ";

    // Printing elements of the array
    for (const string str : strArray) {
        std::cout << str << " ";
    }

    std::cout << std::endl;

    return 0;
}
```

Explanation:

- In this example, a string array named `strArray` is created and initialized with the strings "Apple", "Banana", and "Orange".
- The elements of the array are then printed using a range-based for loop, resulting in the output "String Array: Apple Banana Orange".



String Array:

ARRAY String Array

cpp

Copy code

```
#include <iostream>
#include <string>

int main() {
    // Creating and initializing a string array
    std::string strArray[] = {"Apple", "Banana", "Orange"};
    std::cout << "String Array: ";

    // Printing elements of the array
    for (const auto str : strArray) {
        std::cout << str << " ";
    }

    std::cout << std::endl;

    return 0;
}
```

```
for (const string str : strArray) {
    std::cout << str << " ";
}
```

Explanation:

- In this example, a string array named `strArray` is created and initialized with the strings "Apple", "Banana", and "Orange".
- The elements of the array are then printed using a range-based for loop, resulting in the output "String Array: Apple Banana Orange".



String Array:

ARRAY String Array

cpp

Copy code

```
#include <iostream>
#include <string>

int main() {
    // Creating and initializing a string array
    std::string strArray[] = {"Apple", "Banana", "Orange"};
    std::cout << "String Array: ";

    // Printing elements of the array
    for (const auto& str : strArray) {
        std::cout << str << " ";
    }

    std::cout << std::endl;

    return 0;
}
```

```
for (const auto str : strArray) {
    std::cout << str << " ";
}
```

Explanation:

- In this example, a string array named `strArray` is created and initialized with the strings "Apple", "Banana", and "Orange".
- The elements of the array are then printed using a range-based for loop, resulting in the output "String Array: Apple Banana Orange".



String Array:

ARRAY String Array

Use of Reference:

Range based loops creates a *copy* of the element for each iteration. Unless you actually need a copy, there is no need to perform it. And if the objects you are copying are any larger than a built-in type (integer, pointer etc.), there is a potential performance penalty.

Ref:

<https://blog.knatten.org/2013/10/13/prefer-using-references-with-range-based-for-loops/>

```
cpp                                                                    Copy code

#include <iostream>
#include <string>

int main() {
    // Creating and initializing a string array
    std::string strArray[] = {"Apple", "Banana", "Orange"};
    std::cout << "String Array: ";

    // Printing elements of the array
    for (const auto& str : strArray) {
        std::cout << str << " ";
    }

    std::cout << std::endl;

    return 0;
}
```

```
for (const auto str : strArray) {
    std::cout << str << " ";
}
```

Explanation:

- In this example, a string array named `strArray` is created and initialized with the strings "Apple", "Banana", and "Orange".
- The elements of the array are then printed using a range-based for loop, resulting in the output "String Array: Apple Banana Orange".

cpp

FOR_EACH

 Copy code

```
#include <iostream>
#include <algorithm>

int main() {
    int myArray[] = {1, 2, 3, 4, 5};

    // Use std::for_each with a lambda function to iterate through the array
    std::for_each(std::begin(myArray), std::end(myArray), [](int value) {
        std::cout << value << "\n";
    });

    return 0;
}
```

```
//signature
for_each(
    begin_of_array,
    end_of_array,
    lambda_function
)
```

VECTORS

Unit-01.4

Include the necessary header:

cpp

 Copy code

```
#include <vector>
```

Declaration and Initialization:

cpp

 Copy code

```
// Declare an empty vector of integers
std::vector<int> myVector;

// Declare and initialize with values
std::vector<int> myVector = {1, 2, 3, 4, 5};
```

Accessing Elements:

cpp

 Copy code

```
// Access an element by index
int element = myVector[2];

// Access an element using at() (with bounds checking)
int element = myVector.at(2);

// Access the first and last elements
int firstElement = myVector.front();
int lastElement = myVector.back();
```

Iterating Through a Vector:

cpp

 Copy code

```
// Using a traditional for loop
for (size_t i = 0; i < myVector.size(); ++i) {
    // Access myVector[i]
}

// Using a range-based for loop
for (int value : myVector) {
    // Access value
}

// Using iterators
for (auto it = myVector.begin(); it != myVector.end(); ++it) {
    // Access *it
}
```

Adding Elements:

cpp

 Copy code

Add an element at the end

```
myVector.push_back(6);
```

Insert an element at a specific position

```
myVector.insert(myVector.begin() + 2, 99);
```

Append elements from another vector

```
myVector.insert(myVector.end(), anotherVector.begin(), anotherVector.end())
```

Removing Elements:

cpp

 Copy code

 Copy code

```
// Remove the last element
myVector.pop_back();

// Erase an element at a specific position
myVector.erase(myVector.begin() + 2);

// Erase a range of elements
myVector.erase(myVector.begin() + 1, myVector.begin() + 4);

// Remove all elements with a specific value
myVector.erase(std::remove(myVector.begin(), myVector.end(), 3), myVector.end());
```

Size and Capacity:

cpp

 Copy code

```
// Get the number of elements
size_t size = myVector.size();

// Check if the vector is empty
bool isEmpty = myVector.empty();

// Resize the vector
myVector.resize(10);

// Get the capacity (total available storage)
size_t capacity = myVector.capacity();

// Reserve additional capacity
myVector.reserve(20);
```

Sorting:

cpp

 Copy code

```
// Sort the vector in ascending order
std::sort(myVector.begin(), myVector.end());

// Sort in descending order
std::sort(myVector.begin(), myVector.end(), std::greater<int>());
```

Other Useful Functions:

cpp

 Copy code

```
// Clear all elements
myVector.clear();

// Swap two vectors
myVector.swap(anotherVector);

// Find the first occurrence of a value
auto it = std::find(myVector.begin(), myVector.end(), 5);
if (it != myVector.end()) {
    // Found
}

// Check if a value exists in the vector
bool exists = std::find(myVector.begin(), myVector.end(), 7) != myVector.end();
```

LIST

Unit-01.5

Include the necessary header:

cpp

 Copy code

```
#include <list>
```

Declaration and Initialization:

cpp

 Copy code

```
// Declare an empty list of integers  
std::list<int> myList;  
  
// Declare and initialize with values  
std::list<int> myList = {1, 2, 3, 4, 5};
```

Accessing Elements:

cpp

 Copy code

```
// Lists do not support direct access by index
// Access an element using iterators
auto it = myList.begin();
int firstElement = *it;

// Access the first and last elements
int firstElement = myList.front();
int lastElement = myList.back();
```

Iterating Through a List:

cpp

 Copy code

```
// Using iterators
for (auto it = myList.begin(); it != myList.end(); ++it) {
    // Access *it
}

// Using a range-based for loop (C++11 and later)
for (int value : myList) {
    // Access value
}
```

Adding Elements:

cpp

 Copy code

```
// Add an element at the end
myList.push_back(6);

// Add an element at the beginning
myList.push_front(0);

// Insert an element at a specific position
auto it = std::next(myList.begin(), 2); // Iterator pointing to the thi
myList.insert(it, 99);

// Append elements from another list
myList.insert(myList.end(), anotherList.begin(), anotherList.end());
```

Removing Elements:

cpp

 Copy code

```
// Remove the last element
myList.pop_back();

// Remove the first element
myList.pop_front();

// Erase an element at a specific position
auto it = std::next(myList.begin(), 2); // Iterator pointing to the third element
myList.erase(it);

// Erase a range of elements
auto first = std::next(myList.begin(), 1);
auto last = std::next(myList.begin(), 4);
myList.erase(first, last);

// Remove all elements with a specific value
myList.remove(3);
```

Size and Capacity:

cpp

 Copy code

```
// Get the number of elements
size_t size = myList.size();

// Check if the list is empty
bool isEmpty = myList.empty();
```

Sorting:

cpp

 Copy code

```
// Lists have their own sort member function  
myList.sort(); // Sort in ascending order  
  
// Sort in descending order  
myList.sort(std::greater<int>());
```

Other Useful Functions:

cpp

 Copy code

```
// Clear all elements
```

```
myList.clear();
```

```
// Swap two lists
```

```
myList.swap(anotherList);
```

```
// Reverse the order of elements
```

```
myList.reverse();
```

```
// Splice elements from another list
```

```
auto it = std::next(anotherList.begin(), 2);
```

```
myList.splice(myList.begin(), anotherList, it, anotherList.end());
```

STACK

Unit-01.6

Include the necessary header:

cpp

 Copy code

```
#include <stack>
```

Declaration and Initialization:

cpp

 Copy code

```
// Declare an empty stack of integers
std::stack<int> myStack;

// Declare and initialize with values
std::stack<int> myStack({1, 2, 3, 4, 5});
```

Push and Pop Operations:

cpp

 Copy code

```
// Push an element onto the stack  
myStack.push(6);  
  
// Pop the top element from the stack  
myStack.pop();
```

Accessing the Top Element:

cpp

 Copy code

```
// Access the top element (without removing it)  
int topElement = myStack.top();
```

Checking if the Stack is Empty:

cpp

 Copy code

```
// Check if the stack is empty  
bool isEmpty = myStack.empty();
```

Size of the Stack:

cpp

 Copy code

```
// Get the number of elements in the stack  
size_t size = myStack.size();
```

Checking if the Stack is Empty:

cpp

 Copy code

```
// Check if the stack is empty  
bool isEmpty = myStack.empty();
```

Size of the Stack:

cpp

 Copy code

```
// Get the number of elements in the stack  
size_t size = myStack.size();
```

Iterating Through a Stack:

cpp

 Copy code

```
// Note: Stacks do not support iterators by default.  
  
// To iterate, pop elements until the stack is empty  
while (!myStack.empty()) {  
    int element = myStack.top();  
    // Process the element  
    myStack.pop();  
}
```

Other Useful Functions:

cpp

 Copy code

```
// Swap two stacks
std::stack<int> anotherStack;
myStack.swap(anotherStack);

// Clear all elements from the stack
while (!myStack.empty()) {
    myStack.pop();
}
```

COMPARISONS

Unit-01.7

Criteria	Array	Vector	List	Stack
Access Method	Constant-time	Constant-time	Iteration	LIFO (Top)
Size	Fixed	Dynamic	Dynamic	Dynamic
Dynamic Resizing	No	Yes	Yes	Yes
Contiguous Memory Layout	Yes	Yes	No	Yes
Memory Overhead	Minimal	Slightly higher	Higher	Moderate
Insertions/Deletions	Inefficient	Efficient but may involve copying	Efficient at any position	Efficient push/pop
Direct Access by Index	Yes	Yes	No	No
Suitable Use Cases	Known size; constant-time access	Dynamic size; efficient push/pop	Dynamic size; efficient insertions/deletions	LIFO order; function calls, undo mechanisms

MAP

Unit-01.8

cpp

```
#include <iostream>
#include <map>

int main() {
    // Declare a map with int keys and string values
    std::map<int, std::string> myMap;
```

```
    // Insert key-value pairs
    myMap[1] = "One";
    myMap[2] = "Two";
    myMap[3] = "Three";

    // Access values using keys
    std::cout << "Value at key 2: " << myMap[2] << std::endl;

    // Check if a key is present
    if (myMap.find(4) != myMap.end()) {
        std::cout << "Value at key 4: " << myMap[4] << std::endl;
    } else {
        std::cout << "Key 4 not found in the map." << std::endl;
    }

    // Iterate over the map
    std::cout << "Iterating over the map:" << std::endl;
    for (const auto& pair : myMap) {
        std::cout << "Key: " << pair.first << ", Value: " << pair.second
        << std::endl;
    }

    return 0;
}
```